Enterprise Document Intelligence Assistant

System Architecture & Processing Workflow: A Guide for Business Leaders

Executive Summary: Documents are the lifeblood of modern enterprise, but manual data extraction and reading is slow, expensive, and error-prone. The *Enterprise Document Intelligence Assistant* is a state-of-the-art software engine built to automatically ingest, read, interpret, and securely structure unstructured files (such as PDFs, scans, Word files, Excel files, and emails). By converting plain files into highly searchable and AI-ready information segments (vectors), it empowers organizations to instantly search, query, and unlock insights from their internal knowledge base.

The Core Challenge & Our Solution

For non-technical readers, standard computers view files like PDFs or spreadsheets simply as pixels on a screen or massive unstructured blocks of text. Traditional search engines can search for exact words, but they don't *understand context*. For instance, searching for 'agreements' might miss a document titled 'Service Level SLA'.

This platform solves this by running an intelligent, assembly-line-like ingestion pipeline. Whenever an organization member uploads a document, the system does not just save the file. It opens it, reads scanned images using artificial sight (OCR), cuts the document into digestible pieces, and translates those pieces into a sequence of mathematical numbers (called 'embeddings') that capture the deep conceptual meaning of the sentences. This allows AI models to later locate the exact answer to questions in seconds.

System Architecture Overview

To ensure the system remains lighting-fast even when processing millions of pages, the architecture is divided into three primary components, operating like a high-end restaurant:

1. The Receptionist (FastAPI Web Server)	2. The Kitchen Crew (Celery & Redis)	3. The Archive Vault (PostgreSQL & pgvector)
This handles the user interface requests. It manages security (who is logged in, who owns what project), accepts uploads, and displays results. Because it needs to respond instantly, it never performs heavy document reading itself. Instead, it registers the file and immediately hands the hard work to the kitchen crew.	Document processing is heavy, slow work. To prevent the server from freezing, this system uses *Celery*. It is a background worker network. *Redis* acts as the messaging conveyor belt. The server places a processing task on the belt, and Celery workers pick it up and process it silently in the background.	Once text is extracted and analyzed, it must be stored securely. We use *PostgreSQL*, an enterprise-grade database. Specifically, we equip it with a module called *pgvector*. This acts like a spatial map of concepts, storing coordinates (vectors) for sentences so we can find conceptually similar texts instantly.

Visual Workflow of Information Flow

Inside the Document Processing Pipeline

When the Celery worker handles a document, it routes it through a specialized pipeline with 5 stages. Here is what happens under the hood in plain terms:

1. Security & Integrity Scan

The system conducts file integrity checks to confirm that the file is not corrupt and simulates a malware/virus scan. This prevents malicious uploads from contaminating database services.

2. Text & Layout Extraction (Reading)

This is handled by our **Document Extractor**. Files can take many shapes: spreadsheets, presentations, scanned images, text files, emails, or even compressed ZIP folders. The extractor has specific custom readers for each type. For example:
• *Word Documents:* Paragraphs are read chronologically, and cell content is extracted from tables.
• *Excel Sheets:* Grids are converted into readable tables, omitting completely blank rows.
• *Emails (.eml):* Headers (Sender, Recipient, Subject, Date) are preserved alongside the email body.

3. Optical Character Recognition (OCR Failsafe)

Often, documents are scanned images or static PDFs containing no selectable text. If the system detects a PDF has very low text density, it automatically triggers our **OCR Service**. This service supports industry-standard neural sight frameworks like *Tesseract*, *EasyOCR*, and *PaddleOCR*. It rasterizes pages into crisp, high-resolution images in memory, reads the text within them, and feeds it back into the pipeline.

4. Smart Text Chunking (Slicing)

Large documents cannot be fed into AI search indices all at once due to memory and context constraints. Our **Chunker Service** slices documents into manageable sections using customizable strategies:
• *Fixed-Size Strategy:* Splits text into segments containing an exact number of word-tokens, with a minor overlapping margin so context isn't lost at the border.
• *Sentence-Aware Strategy:* Cuts text at sentence boundaries (periods, exclamation marks) so thoughts are kept whole, rather than splitting sentences in half.
• *Markdown Strategy:* Slices the document based on headers (# Titles, ## Sections), mapping layout structures naturally.

5. Generating Concept Vectors (Brain Work)

Finally, we run the chunks through the **Embedding Service**. Words are converted into high-dimensional numerical vectors. Two sentences that mean similar things but use entirely different words (e.g., 'Company income increased' vs. 'Acme revenue grew') will generate vectors that are geometrically close in mathematical space. The system supports leading AI providers like *OpenAI* (text-embedding-3-small) and local models like *HuggingFace*, with a mock generator for offline development.

Key Benefits for Business & Strategy

Understanding why these architecture decisions were made is crucial for evaluating its business impact:

Multi-Format Standardisation	Instead of using separate systems for emails, spreadsheets, and PDFs, this assistant normalizes all incoming information into a single standard chunk format. Employees can search across all file formats with a single query.
Intelligent Context Recall	Because the database is built around pgvector, search is based on *meaning* rather than keywords. This saves hundreds of hours of manual document review by returning the exact paragraph answers to complex questions.
Failsafe Robustness	The code features extensive custom checks and automatic fallbacks. If the system fails to read text directly from a PDF, it falls back to OCR. If configured AI services are offline, it falls back to local models or mock placeholders to prevent system crashes, maximizing system uptime.
Asynchronous Efficiency	Processing large documents takes time. By queueing jobs, the web app feels fast and instantaneous to users. They upload their files, see a 'Processing' status, and can perform other tasks while the work is done in the background.
Enterprise Boundaries	The system segregates documents cleanly into 'Projects' under 'Organizations', which prevents cross-tenant data leaks. Audit logs keep track of every action, fulfilling enterprise compliance and security standards.

Note on Document Access: This PDF was auto-generated to reflect the codebase's architecture dynamically. To change system settings (such as replacing the OCR engine or adding OpenAI keys), developers can modify the system environment configuration file (.env) or contact the IT Infrastructure lead.